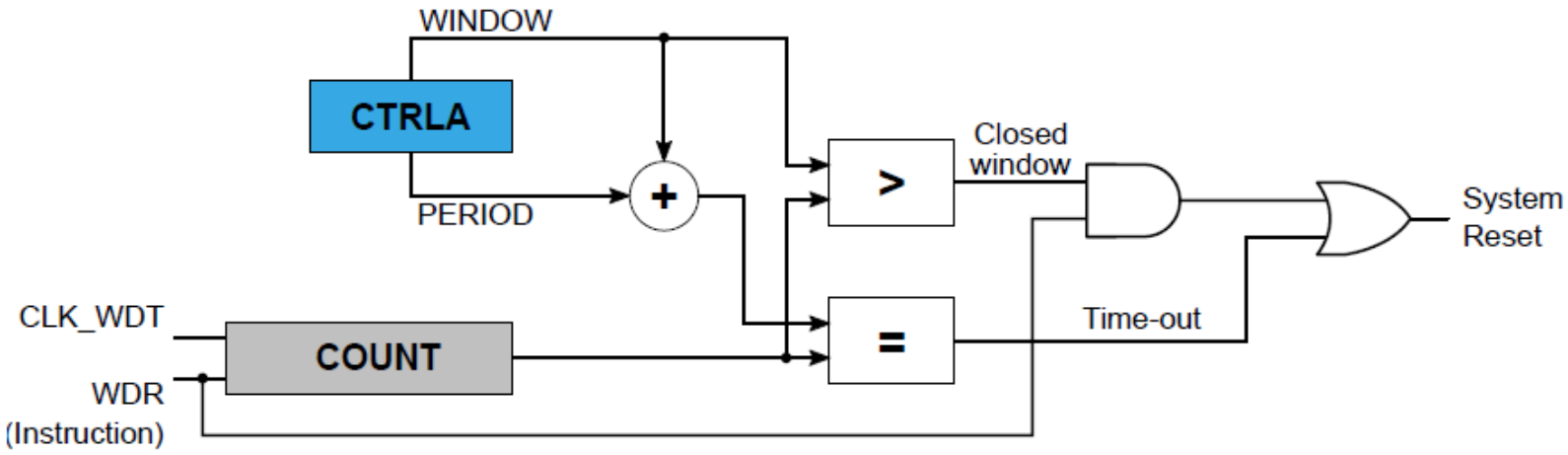
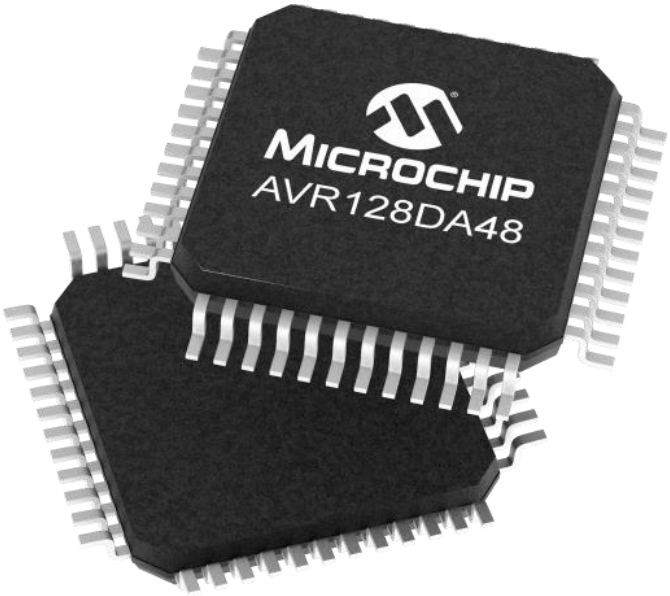
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 24 - Watchdog Timer (WDT)





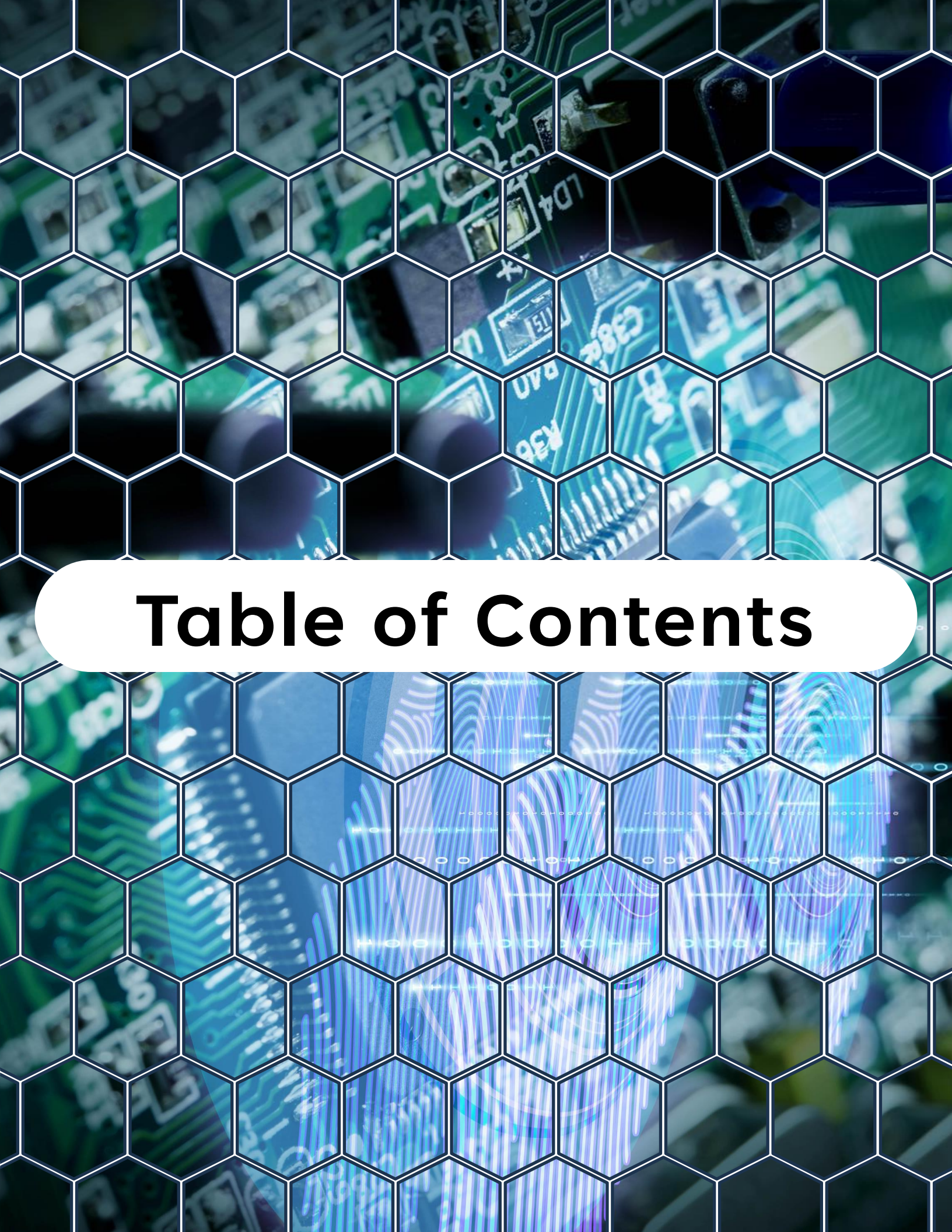
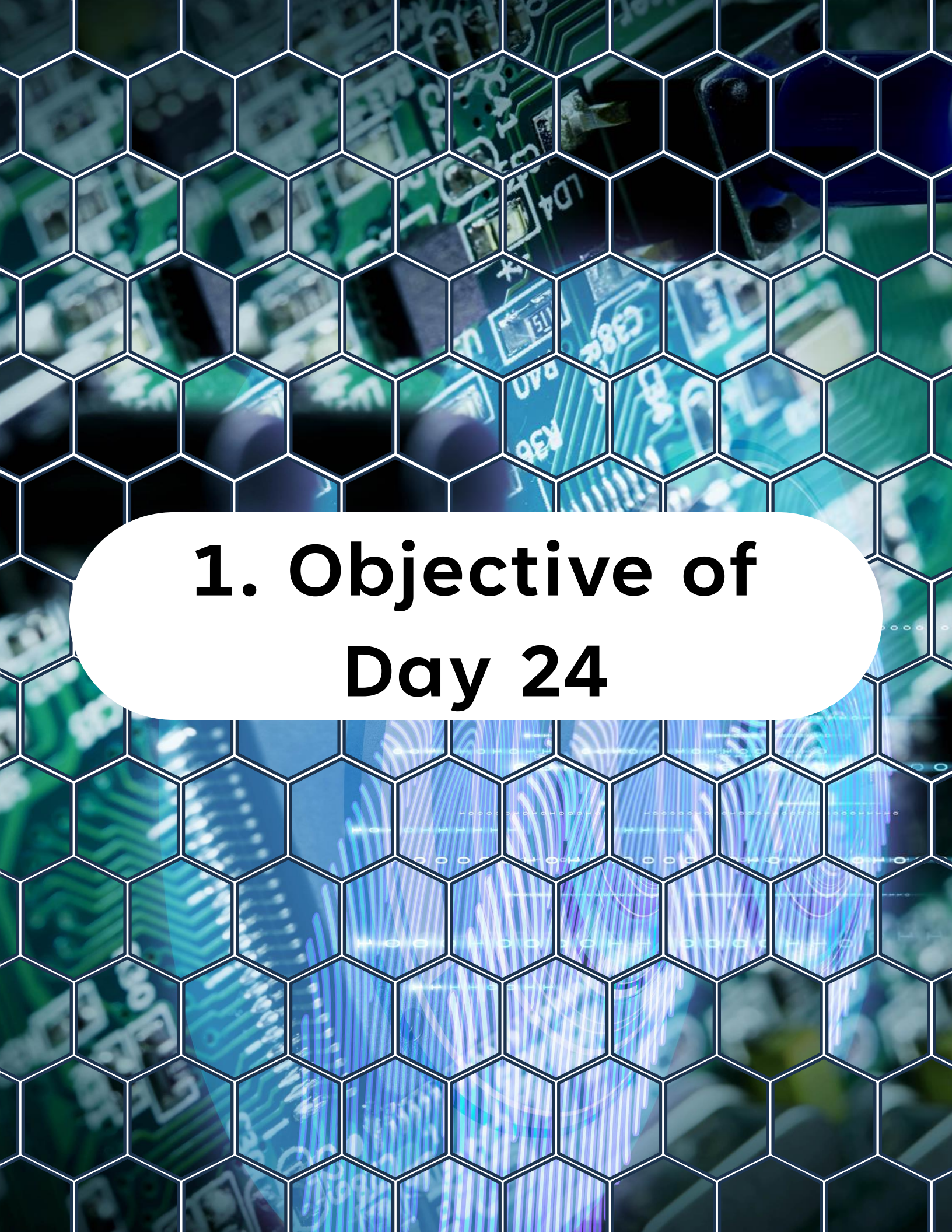


Table of Contents

Table of Contents



1. Objective of Day 24

1. Objective of Day 24

In real embedded systems, software failures are not theoretical. They happen.

A firmware bug, unexpected state, memory corruption, or EMI disturbance can lock the CPU in an infinite loop. When that happens, the system may stop responding permanently.

The Watchdog Timer (WDT) is a hardware safety mechanism designed to detect this condition and automatically reset the microcontroller.

1. Objective of Day 24

By the end of today, you will understand:

- What the Watchdog Timer is and why it exists
- How the WDT works internally
- How to configure timeout periods
- How to reset (service) the watchdog
- How to design firmware that safely uses the WDT

The lab will build a simple system that intentionally fails and demonstrates automatic system recovery.

2. What Is a Watchdog Timer?

2. What Is a Watchdog Timer?

The Watchdog Timer is a hardware timer that runs independently of the CPU.

If the CPU fails to periodically reset the watchdog counter, the watchdog assumes the system is stuck and forces a reset.

The concept is simple:

System running normally

- Firmware periodically resets watchdog

If firmware stops responding

- Watchdog timeout occurs
- MCU reset

This mechanism is critical in safety-critical and unattended systems.

3. Why Watchdogs Are Important?

3. Why Watchdogs Are Important?

Many real systems operate without human supervision:

- Industrial controllers
- IoT sensors
- Automotive electronics
- Power supply controllers
- Aviation electronics

If firmware crashes, the system must recover automatically.

The watchdog guarantees the system will restart if the firmware stops executing correctly.

4. Watchdog Architecture in AVR128DA48

4. Watchdog Architecture in AVR128DA48

The Watchdog Timer operates from its own internal oscillator.

Important characteristics:

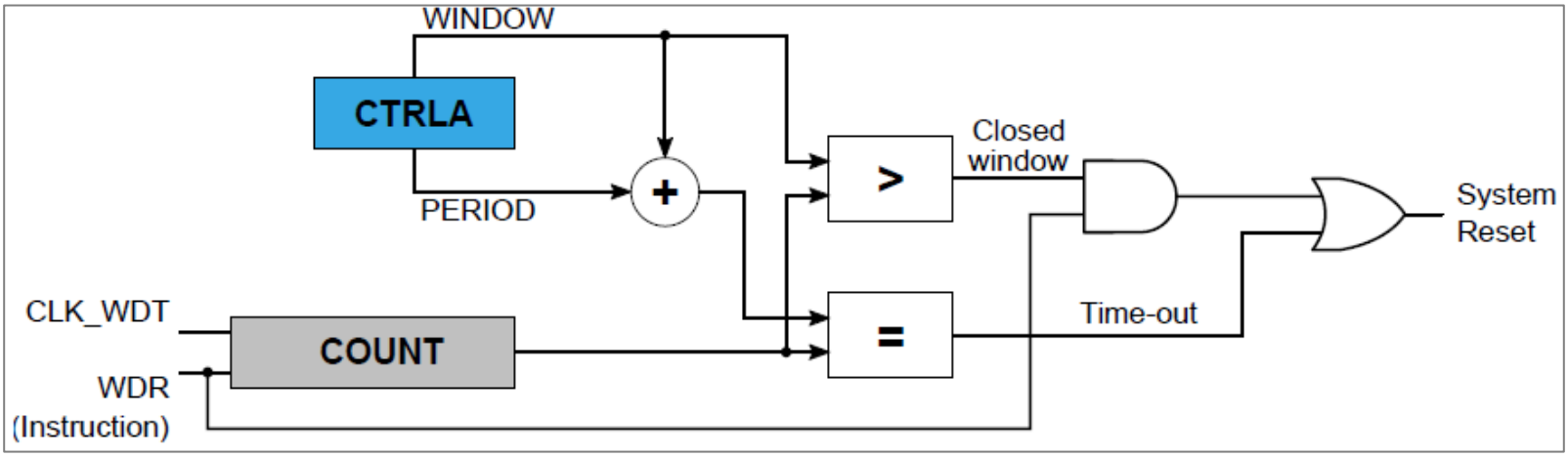
- Runs independently from the CPU clock
- Continues operating in sleep modes
- Cannot be stopped accidentally once properly configured
- Can generate either an interrupt or a reset

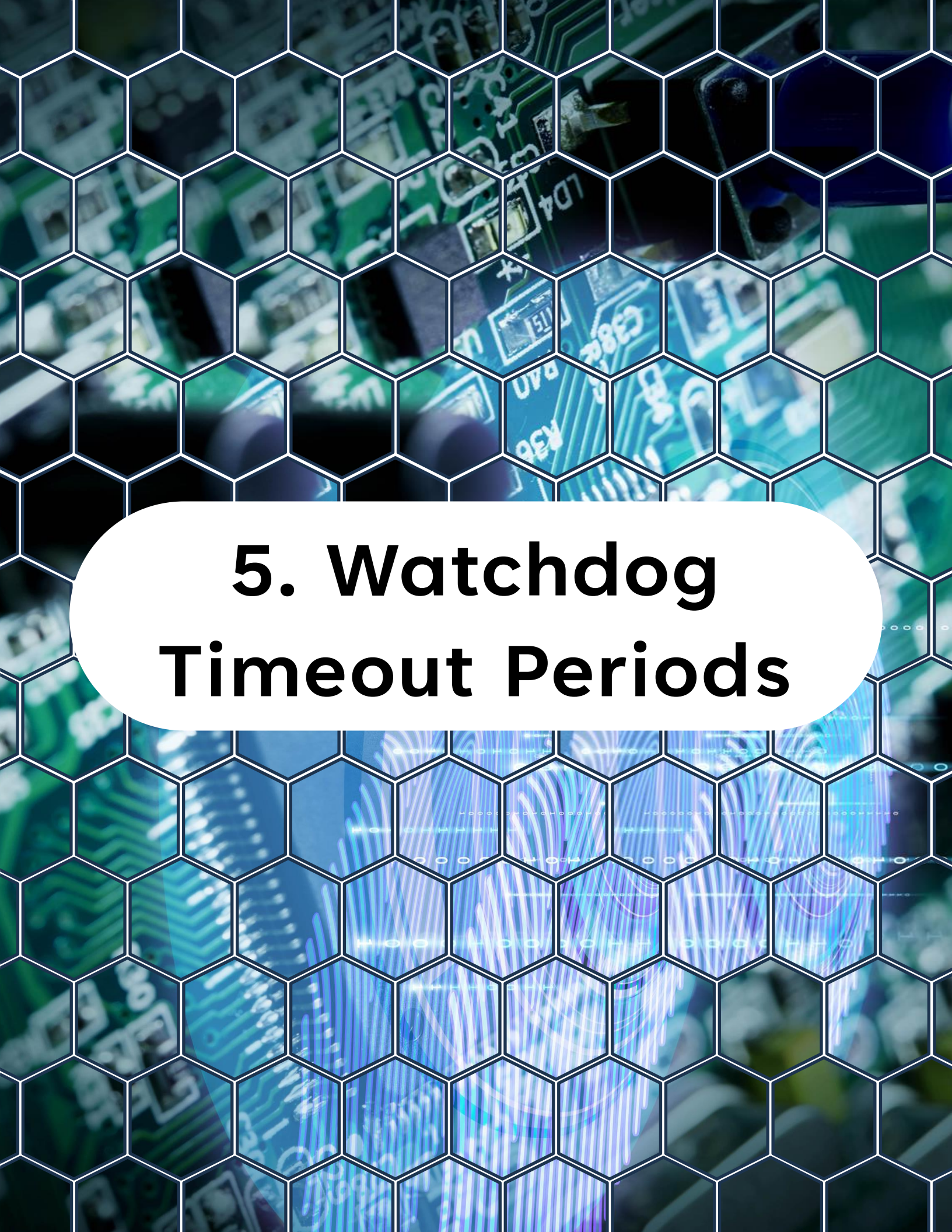
Key register:

WDT.CTRLA

4. Watchdog Architecture in AVR128DA48

WDT Block Diagram





5. Watchdog Timeout Periods

5. Watchdog Timeout Periods

The watchdog timeout determines how long the system has to reset the watchdog before a reset occurs.

Typical timeout values include:

- 8 ms, 16 ms, 32 ms, 64 ms, 125 ms, 250 ms, 500 ms, 1 s, 2 s, 4 s, and 8 s.

The correct timeout depends on the application.

Example considerations:

If the main loop runs every 50 ms, a 500 ms watchdog is reasonable.

6. Enabling the Watchdog

6. Enabling the Watchdog

The watchdog is configured using

WDT.CTRLA

Example configuration: 1-second timeout

```c
/**
 * @brief CCP write helper function
 *
 * @param addr Address of the protected IO register to write to
 * @param value Value to write to the protected IO register
 *
 * @return void
 */
static inline void ccp_write_io(volatile uint8_t *addr, uint8_t value)
{
    CCP = CCP_IOREG_gc;   // unlock protected IO regs for a few cycles
    *addr = value;
}

// Enabling the Watchdog Timer (WDT) with a timeout
// period of 1K clock cycles and no window mode.
ccp_write_io((void*)&(WDT.CTRLA), WDT_PERIOD_1KCLK_gc |
                                  WDT_WINDOW_OFF_gc);
```

Once enabled, the watchdog begins counting immediately.

7. Resetting the Watchdog

7. Resetting the Watchdog

The firmware must periodically reset the watchdog counter.

This operation is called:

- Kicking the watchdog
- Feeding the watchdog
- Servicing the watchdog

The instruction used is:

```
// Reset the watchdog timer to prevent
// it from resetting the system
__asm__ __volatile__("wdr");
```

If this instruction is not executed before the timeout expires, the MCU resets.

8. Detecting a Watchdog Reset

8. Detecting a Watchdog Reset

When the MCU resets, it is often useful to know the cause of the reset.

The reset cause is stored in:

RSTCTRL.RSTFR

Example causes:

- Power-on reset
- Brown-out reset
- External reset
- Watchdog reset
- Software Reset

8. Detecting a Watchdog Reset

Example check:

```
1 // Check the reset cause
2 if (RSTCTRL.RSTFR & RSTCTRL_WDRF_bm)
3 {
4     /* System reset caused by watchdog */
5 }
```

Always clear the flag after reading it.

```
1 // Clear the WDRF flag in the RSTCTRL register
2 // to prevent a reset loop
3 RSTCTRL.RSTFR = RSTCTRL_WDRF_bm;
```

9. Best Practices When Using the Watchdog

9. Best Practices When Using the Watchdog

Improper watchdog usage can cause difficult bugs. Follow these guidelines.

9.1 Reset the Watchdog Only in Known Safe Locations

Do not reset the watchdog randomly.

Only reset it when the system has successfully completed important tasks.

Example:

Main loop completed successfully

- Reset watchdog

9. Best Practices When Using the Watchdog

9.2 Never Reset the Watchdog Inside Interrupts

If an interrupt keeps firing, the watchdog may never expire.

Resetting the watchdog from the main control loop ensures the entire system remains responsive.

9.3 Select a Reasonable Timeout

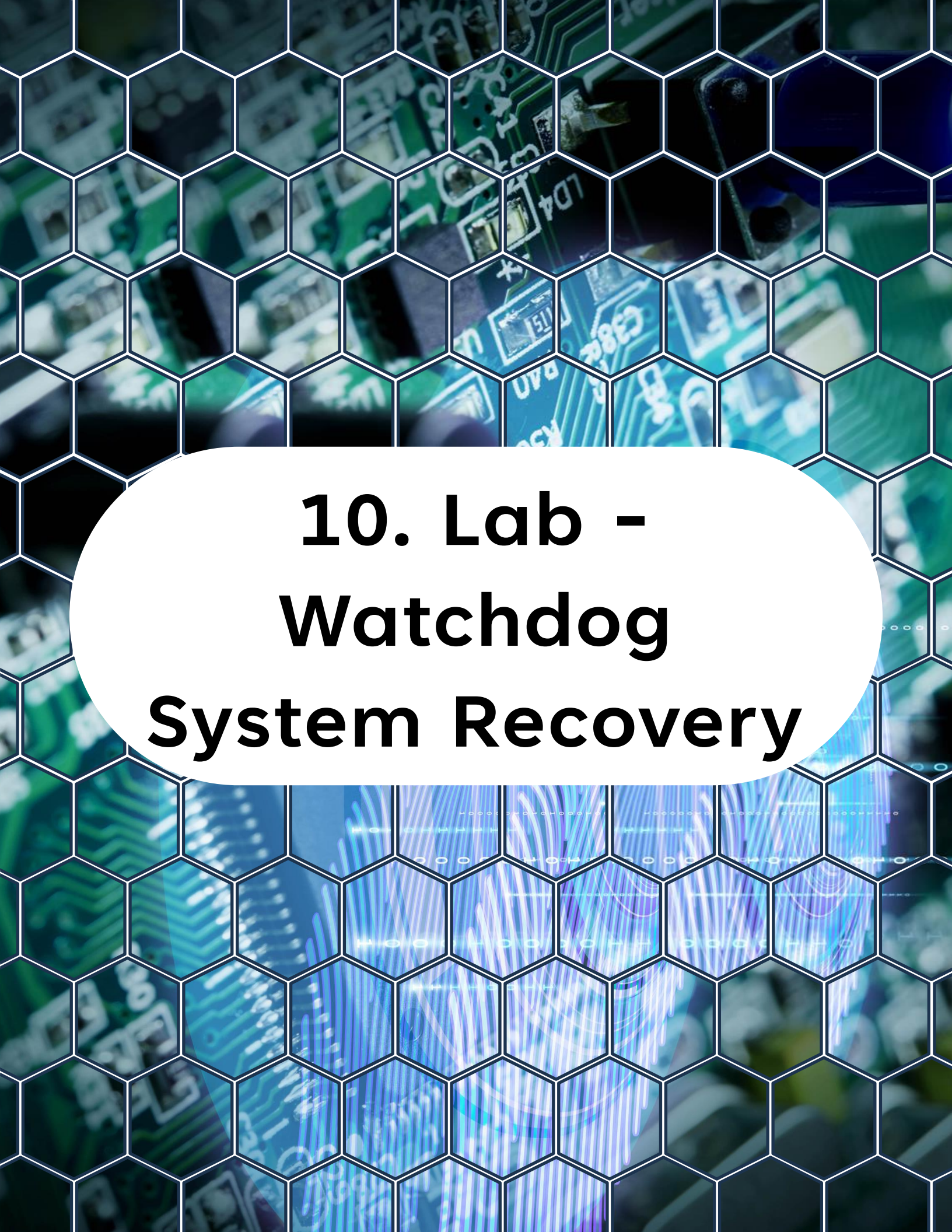
Too short:

- The system resets unnecessarily.

Too long:

- The system may remain frozen for too long.

Choose a timeout slightly longer than the worst-case execution time.



10. Lab - Watchdog System Recovery

10. Lab - Watchdog System Recovery

Goal:

- Enable the watchdog
- Toggle an LED periodically
- Simulate a firmware crash
- Observe automatic MCU reset

10.1 LED Initialization

```c
// Define the LED pin and port.
#define LED_PORT        PORTC
#define LED_PIN_bm      PIN6_bm

static void led_init(void)
{
    // Set the LED pin as an output.
    LED_PORT.DIRSET = LED_PIN_bm;
}
```

10. Lab - Watchdog System Recovery

10.2 Delay Function

```c
static void delay(void)
{
    for (volatile uint32_t i = 0; i < 300000; i++)
    {
    }
}
```

10.3 Watchdog Initialization

```c
static void wdt_init(void)
{
    // Write the desired period; window mode is
    //left OFF (bits 7:4 = 0)
    ccp_write_io((void *)&WDT.CTRLA, WDT_PERIOD_1KCLK_gc);

    // Wait until the register value has
    // synchronized to the ULP domain
    while (WDT.STATUS & WDT_SYNCBUSY_bm)
    {
        /* spin – takes a few ULP cycles
        (~few µs at 4 MHz CPU) */
    }
}
```

10. Lab - Watchdog System Recovery

10.4 Main Program

```c
#include <avr/io.h>
#include <avr/cpufunc.h>   /* ccp_write_io() */

int main(void)
{
    // Initialize the LED pin as an output
    LED_init();

    // Initialize the watchdog timer
    WDT_init();

    while (1)
    {
        // Toggle the LED state
        LED_PORT.OUTTGL = LED_PIN_bm;

        // Wait for a short delay (you can adjust this as needed)
        delay();

        // Reset the watchdog timer to prevent a system reset
        __asm__ __volatile__("wdr");
    }
}
```

The watchdog is reset every loop iteration.

System runs normally.

10. Lab - Watchdog System Recovery

10.5 Simulating a Firmware Failure

Modify the program:

Remove the watchdog reset instruction.

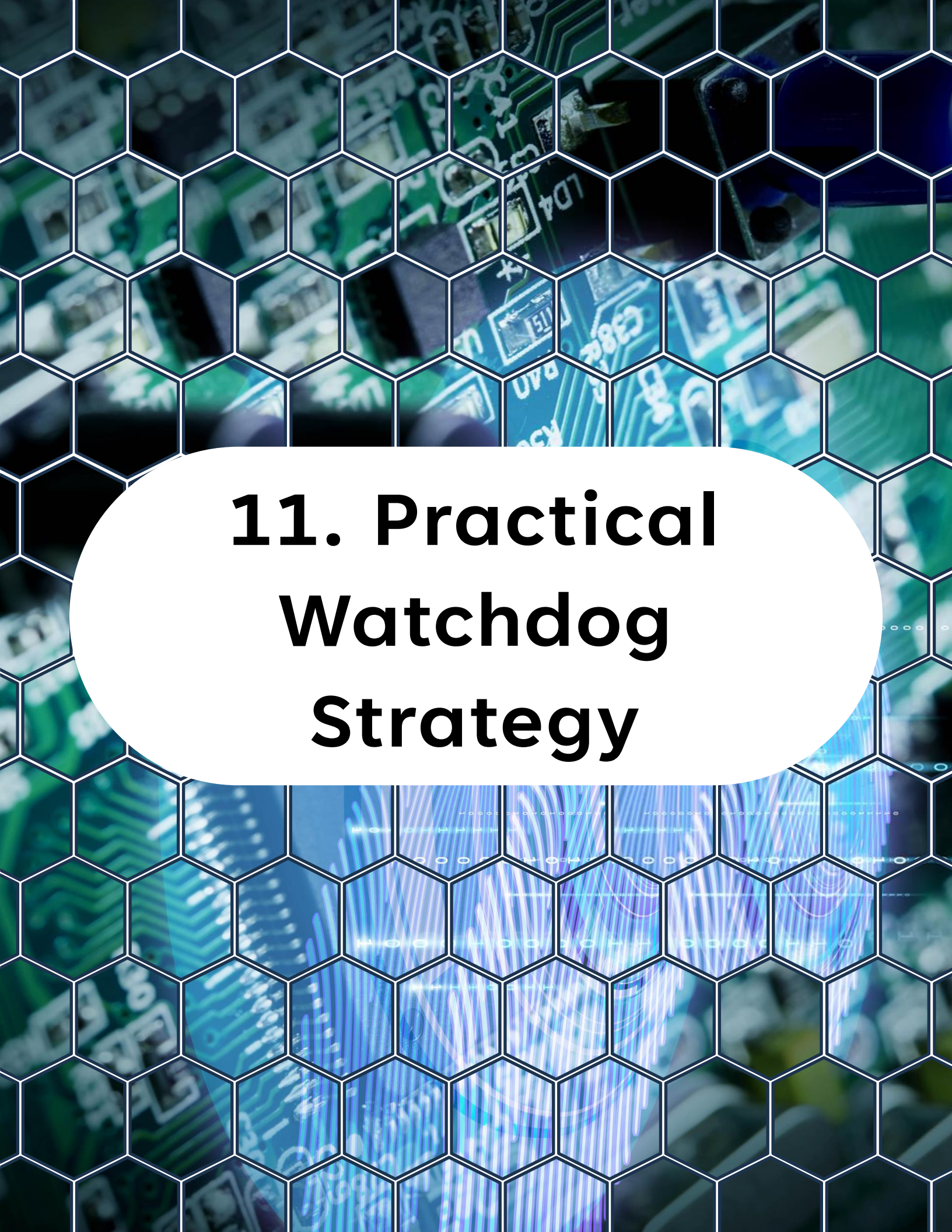
```
while (1)
{
    // Toggle the LED state
    LED_PORT.OUTTGL = LED_PIN_bm;

    // Wait for a short delay (you can adjust this as needed)
    delay();
}
```

Now the watchdog will expire after approximately 1 second. Result:

- The MCU resets repeatedly.
- The LED will blink briefly, then restart continuously.

This demonstrates automatic recovery.



11. Practical Watchdog Strategy

11. Practical Watchdog Strategy

A common design pattern is:

Main control loop:

- Read sensors
- Update control algorithm
- Transmit data
- Check system health

If all operations succeed:

- Reset watchdog.

If firmware hangs anywhere in this process:

- Watchdog expires and resets the MCU.

This prevents permanent lock-ups.

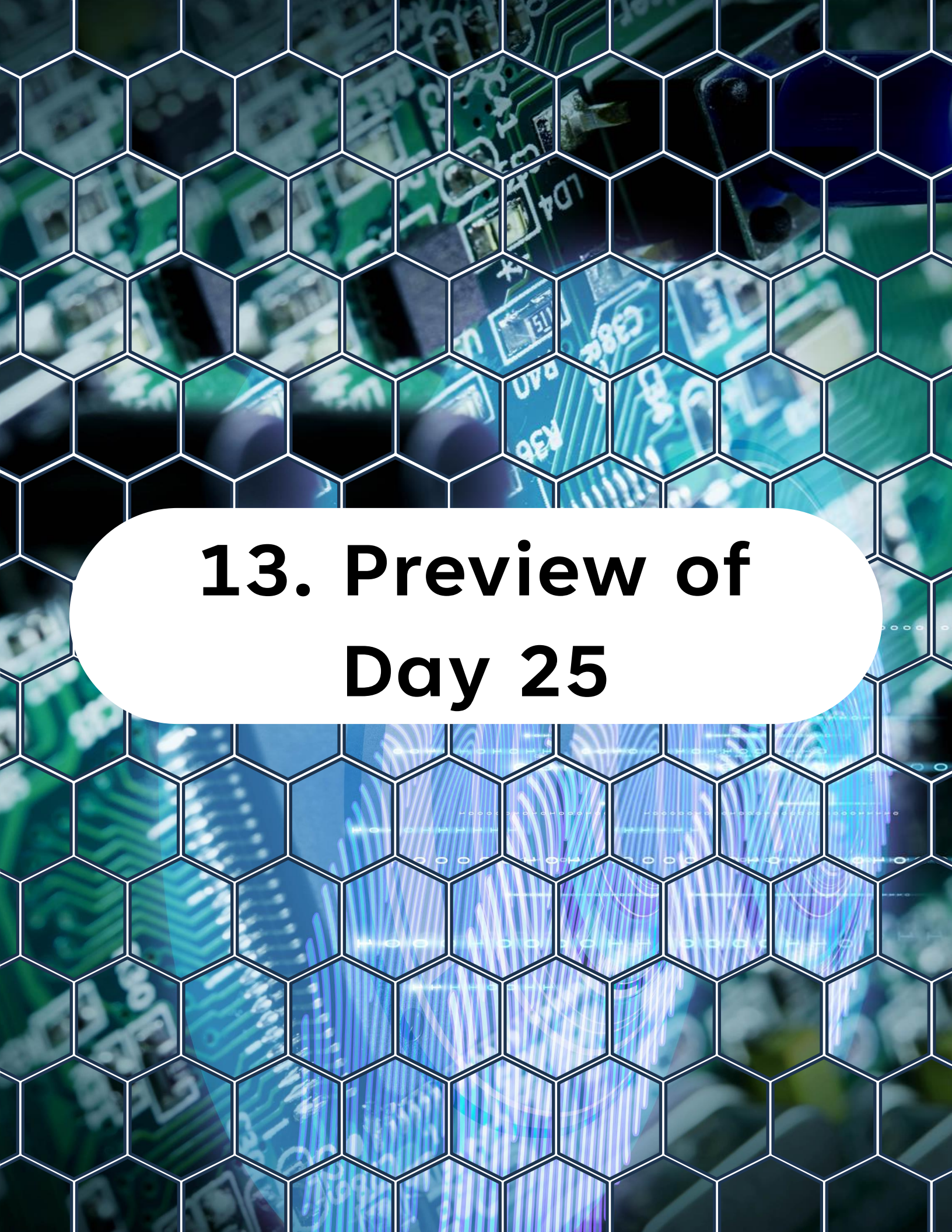
12. What You Should Understand Now

12. What You Should Understand Now

You now know:

- What the Watchdog Timer is
- Why it is essential for reliability
- How to configure watchdog timeouts
- How to reset the watchdog safely
- How to detect watchdog resets

Using a watchdog correctly is one of the most important practices in professional embedded firmware.



13. Preview of Day 25

13. Preview of Day 25

Next we will explore:

Direct Memory Access (DMA) with the AVR DA series.

You will learn how peripherals can move data in memory without CPU involvement, enabling higher performance and lower power consumption.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course